Software Requirements Specification: Disaster Response Resource Allocation System

Field	Detail
Project	Disaster Response Resource Allocation System (DRRAS)
Prepared by	Pratibha Yadav (25MCMC05), Garv Agrawal (25MCMC35)
Prepared for	Prof. Naveen Nekuri, (Software Engineering)
Course	MCA, 3rd Semester
Institution	University of Hyderabad

1. Introduction

1.1 Purpose

This SRS specifies the requirements for the Disaster Response Resource Allocation System (DRRAS). This decision-support tool recommends how many resources to send from each resource base to each affected area. It serves as the reference for design, implementation, and testing, and is written for the project team, the supervisor, and the evaluators.

The system does not make a decision. It produces a suggested allocation and the numbers behind it, and a human coordinator still decides. Its distinguishing feature is that it produces two allocations on identical data, a simple nearest-neighbor baseline and an optimized allocation, and compares them with identical metrics. The project's output is evidence of whether optimization beats the obvious approach, not an allocation in itself.

1.2 Scope

DRRAS covers the following activities.

1.2.1 Scenario data management. The Coordinator enters or loads a disaster scenario: affected areas (location, population, severity, demand) and resource bases (location and the quantity of resources each holds).

1.2.2 Map view. The scenario is shown on a map, with areas coloured by severity and bases as markers, before anything is run.

1.2.3 Travel-time calculation. The system calculates travel time between every base and every area using OSRM once per scenario and stores the result in a travel-time matrix shared by both methods.

1.2.4 Baseline allocation. A greedy method processes areas by severity and sends resources from the nearest base with stock.

1.2.5 Optimized allocation. A constrained optimization model, solved with OR-Tools or PuLP, determines all base-to-area quantities jointly to minimize travel time and unmet demand.

1.2.6 Result comparison. The two stored results are shown side by side, along with response time, unmet demand, and resource use.

Out of scope for this version: machine learning of any kind, integration with real emergency authorities, real-time tracking of vehicles, a mobile app, and user accounts, roles, and authentication. The MVP uses one resource type.

Future work: demand prediction, live weather and traffic feeds, road closures and blocked routes, multiple resource types simultaneously, multi-stage response (waves of deployment over time), and a simulation module.

DRRAS is a prototype built on simulated data. It is not connected to any real emergency service.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
DRRAS	Disaster Response Resource Allocation System
SRS	Software Requirements Specification
Coordinator	The human user who enters scenarios, runs the allocation,s and reads the results
Scenario	One disaster situation: its affected areas and resource bases
Affected area	A location needing help, with population, severity, ty and demand
Severity	How serious the situation in an area is: Low, Medium, High, or Critical, as entered by the Coordinator
Demand	The number of resource units an area needs
Resource base	A location holding resources, for example, an ambulance depot
Resource	The unit being allocated (one resource type in this version)
Allocation	The quantity of resources sent from each base to each area

Term	Definition
Baseline allocation	Allocation by the greedy nearest-available-base method
Optimised allocation	Allocation by the constrained optimization model
Unmet demand	Total resources are short across all areas after allocation
Travel-time matrix	Travel time and distance for every base-area pair
OSRM	Open Source Routing Machine, an external routing service
OSM	OpenStreetMap
OR-Tools, PuLP	Free open-source optimization libraries for Python
PostGIS	Spatial extension for PostgreSQL
DFD	Data Flow Diagram
API, REST	Application Programming Interface, Representational State Transfer
UI	User Interface
MVP	Minimum viable product: the scope built in this version

1.4 References

- DRRAS Feasibility Report (technical, operational, and economic feasibility).
- DRRAS User Stories (product backlog and Sprints 1 to 3).
- DRRAS Data Flow Diagrams, Levels 0, 1, and 2.
- Course textbook notation for use cases and data flow diagrams.
- Documentation of OSRM, OpenStreetMap, OR-Tools, and PuLP.

1.5 Overall Description

DRRAS is a browser-based decision support system. A Coordinator enters a scenario, checks it on a map, and runs two independent allocations on the same data and the same travel-time matrix. The system stores both results and shows them side by side, so the Coordinator can see why the optimized suggestion differs from the obvious answer.

It runs on a standard laptop and uses only free and open-source software. OSRM is the only external system.

2. Overall Description

2.1 Product Perspective

DRRAS is a standalone, web-based application. A React dashboard with a Leaflet.js map talks to a Python FastAPI backend. The backend stores data in PostgreSQL with PostGIS, builds and solves the optimization model using OR-Tools or PuLP, and calls the external OSRM service to compute travel times.

OSRM sits outside the system boundary. Only two things cross that boundary: the Coordinator (sends scenario data, receives results) and OSRM (receives coordinates, returns travel times).

The system keeps three data stores. D1 Scenario Data holds the scenario, areas, bases, and resources. D2 Travel Time Matrix holds the travel time for every base-area pair. D3 Allocation Results holds every run, its assignments, and its metrics. The baseline and optimized methods both read D1 and D2 and write to D3, but they never exchange data.

2.2 Product Functions

ID	Function	Triggered by	DFD process
DR-001	Manage scenario data: enter or load areas, bases, and stock	Coordinator	1.0
DR-002	View map: areas by severity, bases as markers	Coordinator	2.0
DR-003	Run baseline allocation	Coordinator	4.0
DR-004	Run optimized allocation	Coordinator	5.0
DR-005	Compare results side by side	Coordinator	6.0
DR-006	Calculate travel time through OSRM	Neither actor directly; included by DR-003 and DR-004	3.0

DR-006 is a mandatory sub-step that both allocation use cases need before they can run, so it is an <<include>> of DR-003 and DR-004. There is no <<extend>> relationship: every function is either requested by the Coordinator or is a mandatory step of another.

Both methods run on the same scenario and the same travel-time matrix. Any difference between their results, therefore, comes solely from the allocation logic. Running the system on sample scenarios is project testing work, not a use case.

2.3 User Characteristics

Coordinator. The only human user: an emergency planner or coordinator, not a programmer. The Coordinator is expected to:

- Use a web browser and read a map and a small table of numbers;
- Enter or load a scenario, press run,n, and read the result;
- understand severity levels, demand and base stock;
- judge severity, because severity is an input the Coordinator supplies, not something the system measures.

OSRM (Routing Service). A system actor, not a person. It sits outside the system boundary and provides travel-time data upon request.

2.4 User Constraints

- The Coordinator is the only person who touches the system. The system advises, and the Coordinator decides.
- Severity must be Low, Medium, High, or Critical.
- Both allocation methods must run on the same scenario and the same travel-time matrix.
- A base can never send more than it holds, and an area can never receive more than it needs.
- The comparison needs both a baseline run and an optimized run for the same scenario.
- OSRM travel times reflect ordinary traffic and ordinary rules, so they are pessimistic for emergency vehicles.

2.5 Assumptions and Dependencies

Assumptions

- Scenario and resource data are simulated or taken from public datasets. Real base locations and stock levels are not public.
- Every area and base has a location that can be given as coordinates and reached on the road network.
- Demand is the number of resource units an area needs, and base stock is the number of units it holds.
- The MVP handles a handful of areas and bases.
- The resource type is ambulances.
- The Coordinator has a computer with a modern web browser and access to the backend.

Dependencies

- OSRM on OpenStreetMap data is reachable, with a straight-line fallback when it is not.
- PostgreSQL with PostGIS is available.
- OR-Tools or PuLP for optimization.
- React, Leaflet.js, and OpenStreetMap map data for the dashboard.

2.6 Apportioning Requirements

Requirements are delivered in three sprints. Each item depends on the ones before it: the baseline cannot run without the travel-time matrix, which cannot be built without scenario data, which in turn

requires the database. The optimizer comes last because it is the riskiest piece, and by then the data, map, travel times, metrics, and comparison target already exist.

Sprint	Requirements delivered	State at the end of the sprint
1. Setup and scenario data	FR-1, FR-2	A scenario can be entered and viewed on a map. No allocation logic yet.
2. Travel time and baseline	FR-3, FR-4	The system produces a working allocation with real numbers.
3. Optimization and comparison	FR-5, FR-6, FR-7	The optimizer and the side-by-side comparison work on 2 to 3 sample scenarios.

The future work listed in section 1.2 is deferred to later versions.

3. Specific Requirements

3.1 Interface Requirements

3.1.1 User Interface

The UI is a single browser dashboard built with React and Leaflet.js. A person without technical knowledge must be able to use it.

ID	Panel	Requirement
UI-1	Scenario entry	Forms to create a scenario (name, region), add an area (location, population, severity, demand), and add a base (name, location, quantity). Severity is chosen from Low, Medium, High, and Critical.
UI-2	Scenario map	A Leaflet map showing areas colored by severity, with bases as markers and a legend.
UI-3	Run controls	Actions to run the baseline, run the optimized allocation, and compare. Travel times are calculated automatically when they are needed.

ID	Panel	Requirement
UI-4	Result panel	For each run: the assignments (base, area, quantity) and the metrics table.
UI-5	Comparison view	Two maps side by side, one metrics table, the difference in each metric, and which direction is better.
UI-6	Messages	A confirmation after data is saved, a clear error when input is rejected, and a notice when straight-line distances replace OSRM times.
UI-7	Labelling	Simulated data is labeled as such, and results are presented as suggestions for the Coordinator to consider.

3.1.2 Hardware Interface

- No specialized hardware is needed.
- The Coordinator uses any computer that runs a modern web browser.
- No vehicle, sensor, or tracking hardware is connected. Real-time tracking is out of scope.

3.1.3 Software Interface

Interface	Between	Purpose
Web browser	Coordinator and dashboard	Runs the React dashboard and the Leaflet map.
REST API	Dashboard and FastAPI backend	Scenario data, requests, and results. For example, POST /scenarios creates a scenario.
Database	Backend and PostgreSQL with PostGIS	Stores D1, D2, and D3, with spatial types for locations.
Routing service	Backend and OSRM (OpenStreetMap data)	Sends base and area coordinates, receives travel time and distance.
Optimisation library	Backend and OR-Tools or PuLP	Receives the model, returns the solution.

Interface	Between	Purpose
Map data	Leaflet and OpenStreetMap	Supplies the base map without paid map tiles.

3.1.4 Communication Interface

- The dashboard communicates with the backend via HTTP requests that carry JSON (REST).
- The backend requests travel times from OSRM over HTTP, once per scenario.
- If OSRM cannot be reached, the backend falls back to straight-line distance.
- The backend connects to PostgreSQL over a database connection.
- If the system is hosted for a demonstration, traffic uses HTTPS.

3.2 Functional Requirements

Requirements are numbered FR-n.m. The number n matches the DFD process (from FR-1 for process 1.0 to FR-6 for process 6.0), and FR-7 covers requirements that span scenarios. "Shall" marks a mandatory requirement, and "should" a desirable one. Section 4.1 traces every requirement to its use case, DFD process, and user story.

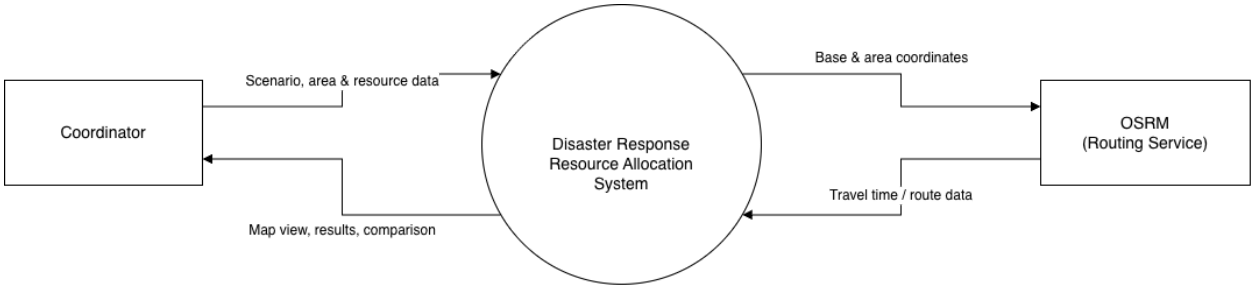
3.2.1 Information Flows

The data flow diagrams describe where the data goes, using three levels.

Notation: external entity = rectangle, process = numbered circle, data store = open-ended rectangle, data flow = labeled arrow.

Level 0: context diagram. The whole system is one process. Only the Coordinator and OSRM cross the boundary.

ID	From	To	Data flow
L0-1	Coordinator	System (0)	Scenario, area & resource data
L0-2	System (0)	Coordinator	Map view, results, comparison
L0-3	System (0)	OSRM	Base & area coordinates
L0-4	OSRM	System (0)	Travel time/routedata

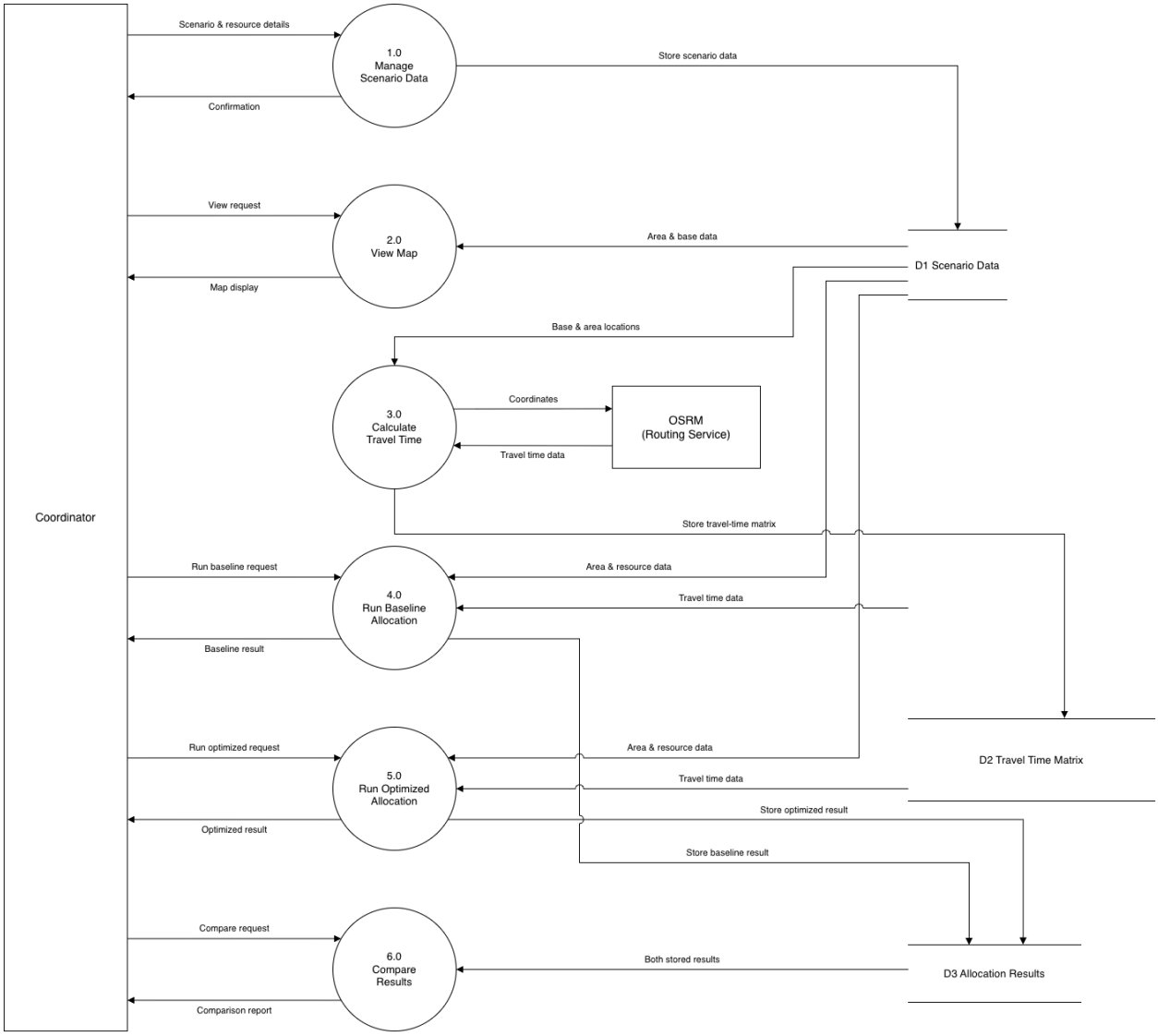


Level 1. The system splits into six processes and three data stores. Only process 3.0 talks to OSRM, which is the same fact as the <<include>> relationship in the use cases. Processes 4.0 and 5.0 never exchange data: both read the same two stores, and both write to D3 separately.

ID	From	To	Data flow
L1-01	Coordinator	1.0 Manage Scenario Data	Scenario & resource details
L1-02	1.0 Manage Scenario Data	Coordinator	Confirmation
L1-03	1.0 Manage Scenario Data	D1 Scenario Data	Store scenario data
L1-04	Coordinator	2.0 View Map	View request
L1-05	2.0 View Map	Coordinator	Map display
L1-06	D1 Scenario Data	2.0 View Map	Area & base data
L1-07	D1 Scenario Data	3.0 Calculate Travel Time	Base & area locations
L1-08	3.0 Calculate Travel Time	OSRM	Coordinates
L1-09	OSRM	3.0 Calculate Travel Time	Travel time data
L1-10	3.0 Calculate Travel Time	D2 Travel Time Matrix	Store travel-time matrix
L1-11	Coordinator	4.0 Run Baseline Allocation	Run baseline request
L1-12	4.0 Run Baseline Allocation	Coordinator	Baseline result
L1-13	D1 Scenario Data	4.0 Run Baseline Allocation	Area & resource data

ID	From	To	Data flow
L1-14	D2 Travel Time Matrix	4.0 Run Baseline Allocation	Travel time data
L1-15	4.0 Run Baseline Allocation	D3 Allocation Results	Store baseline result
L1-16	Coordinator	5.0 Run Optimized Allocation	Run optimized request
L1-17	5.0 Run Optimized Allocation	Coordinator	Optimized result
L1-18	D1 Scenario Data	5.0 Run Optimized Allocation	Area & resource data
L1-19	D2 Travel Time Matrix	5.0 Run Optimized Allocation	Travel time data
L1-20	5.0 Run Optimized Allocation	D3 Allocation Results	Store optimized result
L1-21	Coordinator	6.0 Compare Results	Compare request
L1-22	6.0 Compare Results	Coordinator	Comparison report
L1-23	D3 Allocation Results	6.0 Compare Results	Both stored results

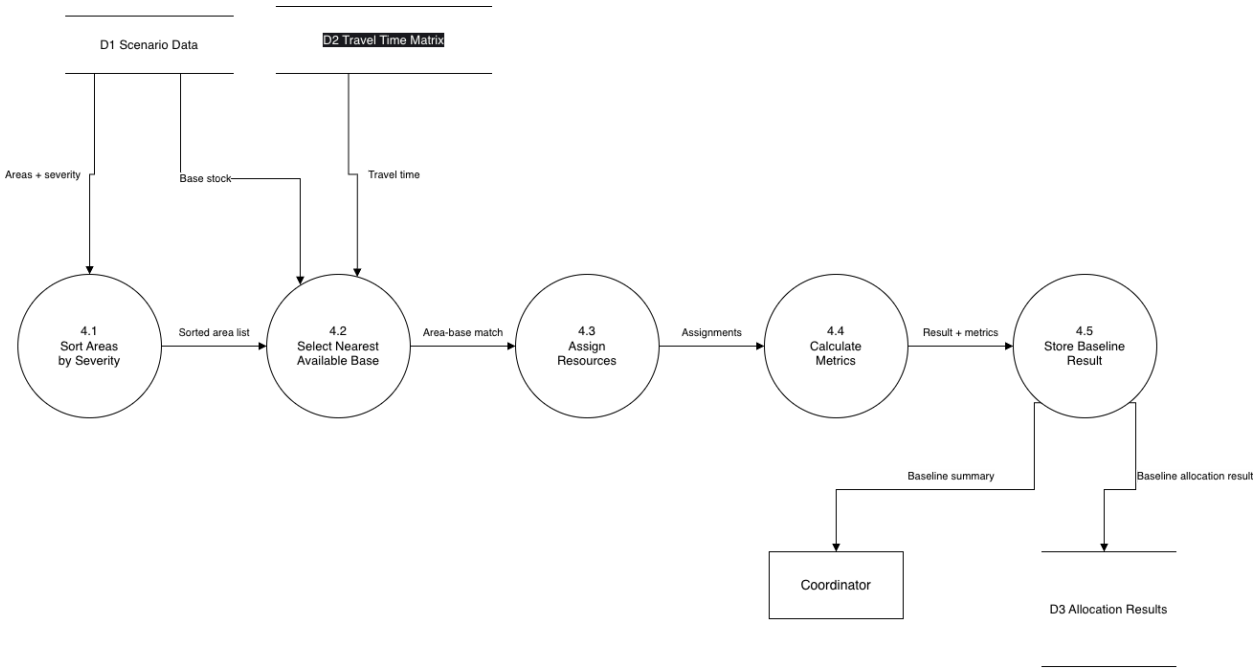
Data store access at Level 1: D1 is written by 1.0 and read by 2.0, 3.0, 4.0, and 5.0. D2 is written by 3.,d read by 4.0 and 5.0. D3 is written by 4.0 and 5.0, and read by 6.0.

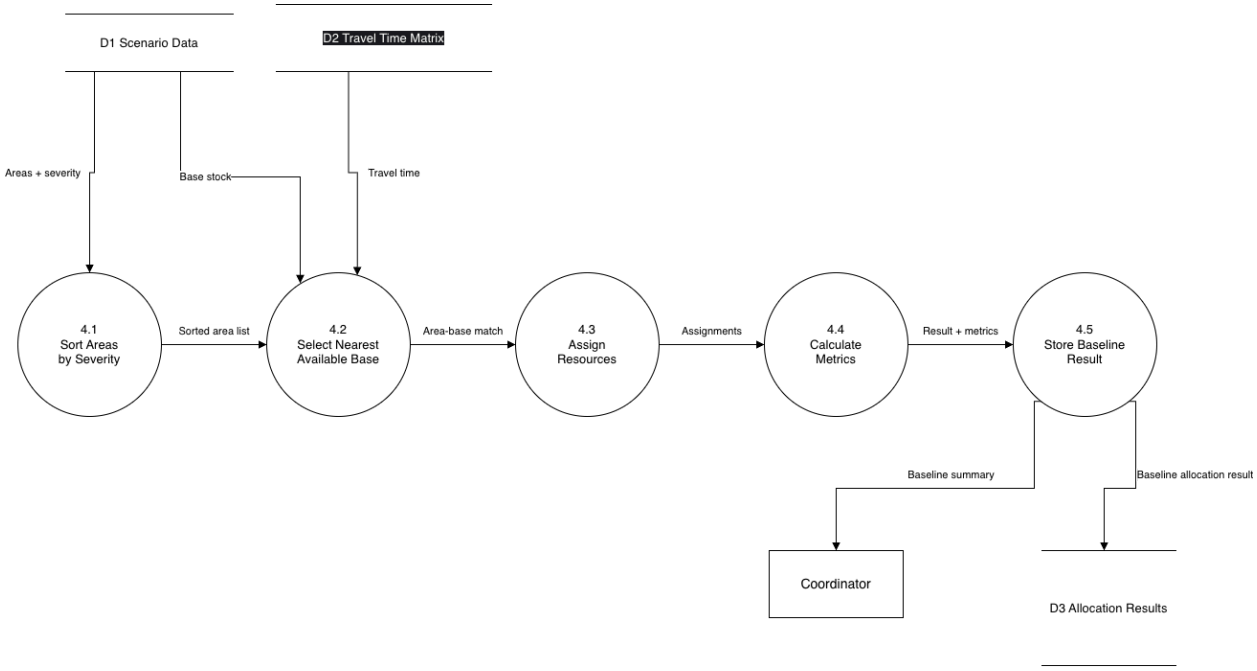


Level 2. Only the two allocation processes are decomposed, because they are the only ones with real internal steps. Both chains end the same way, with metrics and then storage, so the two methods are measured identically.

ID	From	To	Data flow
L4-1	D1 Scenario Data	4.1 Sort Areas by Severity	Areas + severity
L4-2	D1 Scenario Data	4.2 Select Nearest Available Base	Base stock
L4-3	D2 Travel Time Matrix	4.2 Select Nearest Available Base	Travel time

ID	From	To	Data flow
L4-4	4.1 Sort Areas by Severity	4.2 Select Nearest Available Base	Sorted area list
L4-5	4.2 Select Nearest Available Base	4.3 Assign Resources	Area-base match
L4-6	4.3 Assign Resources	4.4 Calculate Metrics	Assignments
L4-7	4.4 Calculate Metrics	4.5 Store Baseline Result	Result + metrics
L4-8	4.5 Store Baseline Result	Coordinator	Baseline summary
L4-9	4.5 Store Baseline Result	D3 Allocation Results	Baseline allocation result
L5-1	D1 Scenario Data	5.1 Build Optimization Model	Areas + resources
L5-2	D2 Travel Time Matrix	5.1 Build Optimization Model	Travel time matrix
L5-3	5.1 Build Optimization Model	5.2 Solve Model (OR-Tools / PuLP)	Variables & constraints
L5-4	5.2 Solve Model (OR-Tools / PuLP)	5.3 Validate Solution	Raw solution
L5-5	5.3 Validate Solution	5.4 Calculate Metrics	Validated allocation
L5-6	5.4 Calculate Metrics	5.5 Store Optimized Result	Result + metrics
L5-7	5.5 Store Optimized Result	Coordinator	Optimized summary
L5-8	5.5 Store Optimized Result	D3 Allocation Results	Optimized allocation result


D1 Scenario Data
D2 Travel Time Matrix
Areas + severity
Base stock
Travel time
4.1
Sort Areas
by Severity
Sorted area list
4.2
Select Nearest
Available Base
Area-base match
4.3
Assign
Resources
Assignments
4.4
Calculate
Metrics
Result + metrics
4.5
Store Baseline
Result
Baseline summary
Baseline allocation result
Coordinator
D3 Allocation Results


D1 Scenario Data
D2 Travel Time Matrix
Areas + severity
Base stock
Travel time
4.1
Sort Areas
by Severity
Sorted area list
4.2
Select Nearest
Available Base
Area-base match
4.3
Assign
Resources
Assignments
4.4
Calculate
Metrics
Result + metrics
4.5
Store Baseline
Result
Baseline summary
Baseline allocation result
Coordinator
D3 Allocation Results

3.2.2 Process Description

1.0 Manage Scenario Data (use case DR-001). Receives scenario and resource details from the Coordinator, stores them in D1, and confirms.

ID	Requirement
FR-1.1	The system shall let the Coordinator create a scenario with a name and region. The creation date is recorded automatically.
FR-1.2	The system shall allow the Coordinator to add affected areas to a scenario, each with a location, population, severity, and demand.
FR-1.3	The system shall reject a severity other than Low, Medium, High, or Critical.
FR-1.4	The system shall allow the Coordinator to add resource bases, each with a name and location, and to record the resource type and quantity held by each base.
FR-1.5	The system shall let the Coordinator enter scenario data or load a prepared sample scenario.
FR-1.6	The system shall validate input before saving: population, demand, and quantity must not be negative, and locations must be valid coordinates.
FR-1.7	The system shall save scenario data in D1 and confirm to the Coordinator that it was saved.

2.0 View Map (DR-002). Reads area and base data from D1 and returns a map display. Nothing is calculated at this step: the Coordinator checks that the scenario looks right before spending time on a run.

ID	Requirement
FR-2.1	The system shall display every area in the selected scenario on a map, colored by severity.
FR-2.2	The system shall show every resource base as a marker.
FR-2.3	The system shall show the map before any allocation is run.

3.0 Calculate Travel Time (DR-006, included by DR-003 and DR-004). Reads base and area locations from D1, exchanges coordinates and travel times with OSRM, and stores the matrix in D2. It is the only process that talks to OSRM.

ID	Requirement
FR-3.1	For every base-area pair of a scenario, the system shall send the coordinates to OSRM and receive the travel time and distance.
FR-3.2	The system shall store the results in D2 as the travel-time matrix, with one row per base-area pair.
FR-3.3	The system shall calculate the matrix once per scenario. Both allocation methods shall reuse it, so they run on identical numbers.
FR-3.4	When an allocation is requested, and the scenario has no matrix yet, the system shall calculate it first.
FR-3.5	If OSRM cannot be reached, the system shall fall back to straight-line distance and notify the Coordinator that it did so.

4.0 Run Baseline Allocation: Reads areas and stock from D1 and travel times from D2, and writes the result to D3. The method is simple on purpose: it is the yardstick the optimizer has to beat.

ID	Requirement
FR-4.1	The system shall sort the affected areas by severity, most severe first.
FR-4.2	For the current area, the system shall select the nearest base that still has resources available, not simply the nearest base.
FR-4.3	The system shall assign as many resources as the base can spare, up to the area's needs, and subtract that amount from the base's remaining stock.
FR-4.4	If the area's demand remains unmet, the system shall repeat from FR-4.2 with the next-nearest base that has stock, then move to the next area until all areas are processed or resources run out.

ID	Requirement
FR-4.5	A base shall never send more than it holds.
FR-4.6	The system shall store the run in D3 with method = baseline, together with its allocations and metrics, and return a baseline summary showing response time and unmet demand.
FR-4.7	The baseline shall run independently of the optimized method and shall not read or change its results.

5.0 Run Optimized Allocation (DR-004; Level 2 processes 5.1 to 5.5). Reads the same D1 and D2 data and writes to D3. Instead of deciding area by area, it solves for every assignment at once.

ID	Requirement
FR-5.1	The system shall read the areas, resources, and travel-time matrix and build the model. The decision variable x[i][j] is the number of resources sent from base i to area j.
FR-5.2	The objective shall minimize a weighted combination of total travel time and total unmet demand.
FR-5.3	The model shall enforce three constraints: a base cannot send more than it has, an area cannot receive more than it needs, and no assignment is negative.
FR-5.4	The system shall solve the model with OR-Tools or PuLP.
FR-5.5	The system shall verify that the returned solution satisfies all constraints before accepting it.
FR-5.6	The system shall calculate the same metrics as the baseline, including solver runtime.
FR-5.7	The system shall store the run in D3 with method = optimized, separately from the baseline run, and return an optimized summary to the Coordinator.
FR-5.8	The optimized method shall run independently of the baseline and shall neither read nor change its results.

6.0 Compare Results (DR-005). Reads both stored runs for a scenario from D3 and returns a comparison report.

ID	Requirement
FR-6.1	The system shall fetch the baseline and optimized runs for the same scenario from D3.
FR-6.2	The system shall show them side by side: two maps and one metrics table.
FR-6.3	The system shall show the difference in each metric and whether a lower or higher value is better.
FR-6.4	The system shall show solver runtime next to allocation quality, so the cost of the better allocation is visible.
FR-6.5	If either run is missing, the system shall tell the Coordinator which one.

Requirements across scenarios

ID	Requirement
FR-7.1	The system shall store several scenarios and keep the results of every run under its scenario.
FR-7.2	The system shall run both methods on each of 2 to 3 sample scenarios without errors and record the results for each.

3.3 Logical Database Requirements

The three data stores map onto eight PostgreSQL tables. Locations use PostGIS spatial types, not loose latitude and longitude numbers. Column types are proposed and may change at design time.

Table	Store	Fields
scenarios	D1	scenario_id (key), name, region, created_at
areas	D1	area_id (key), scenario_id, name, location, population, severity, demand

Table	Store	Fields
resource_bases	D1	base_id (key), scenario_id, name, location
resources	D1	resource_id (key), base_id, resource_type, quantity_available
travel_matrix	D2	base_id, area_id (together the key), travel_time, distance, is_fallback
runs	D3	run_id (key), scenario_id, method, created_at, solver_runtime
allocations	D3	allocation_id (key), run_id, base_id, area_id, quantity_sent
metrics	D3	run_id (key), average_response_time, max_response_time, coverage_ratio, unmet_demand, utilisation, total_travel_cost

Rules the database shall enforce:

- A scenario has many areas and many bases. A base has many resource rows. A scenario has many runs; a run has many allocations; and each run has exactly one metrics row.
- Baseline and optimized results live in the same tables, distinguished by the method column for runs. Compare Results fetches two rows from runs for the same scenario. A third method later needs no schema change.
- Severity is limited to the four allowed values, and population, demand, and quantities are never negative.
- Allocations refer to an existing run, base, and area, and the quantities a run sends from one base never exceed that base's stock.
- travel_matrix holds one row per base-area pair.

3.4 Design Constraints

Every item in the stack is free and open source, and the whole system runs on a normal laptop. The pairing that does the real work is Python with OR-Tools: theoptimizationn model is the heart of the project, and that choice drives the backend language.

Layer	Choice	Reason
Backend	Python + FastAPI	The optimization libraries are in Python, and FastAPI gives quick REST endpoints.
Database	PostgreSQL + PostGIS	Spatial types and queries for locations.
Optimisation	OR-Tools (or PuLP)	Free solvers for the constrained allocation model. PuLP is simpler to read and write.
Routing	OSRM on OpenStreetMap	Real road travel times without a paid maps API.
Frontend	React + Leaflet.js	Leaflet renders the map. React handles the dashboard and comparison view.

The feasibility report also considered Node.js with Express for the backend and MongoDB for the database. The project guide fixes the choices in the table above.

The system shall follow these constraints:

- Run in a web browser, with nothing to install on the Coordinator's side.
- Treat OSRM as an external service, not as a module inside the system.
- Run both allocation methods on the same scenario, the same travel-time matrix, and the same metrics, with neither feeding into the other.
- Store both methods' results in the same tables, distinguished by the method column.
- Label simulated data as simulated, in the report and in the interface.
- Handle one resource type.

3.5 Software System Attributes

3.5.1 Reliability

- The system shall validate input before saving and reject invalid values, such as an unknown severity.
- The baseline shall never send more than the base holds.
- The optimized solution shall be checked against every constraint before it is stored. If the solver returns no valid solution, no optimized run is stored, and the Coordinator is told.
- If OSRM is unreachable, the straight-line fallback shall keep the system usable.
- Every run shall be stored under its scenario so that Compare Results can read it back.
- Stored scenarios and results shall not be lost during normal operation.

3.5.2 Availability

The system is available whenever the team's laptop or the demonstration host is running. It is a decision support prototype, not an emergency operations service, so no uptime target is set.

3.5.3 Security

- The data is simulated and holds no personal information.
- The backend shall validate all input and use parameterized database queries.
- Database credentials shall be kept out of the source code and the front end.
- If the system is hosted for a demonstration, it shall use HTTPS.

3.5.4 Maintainability

The system should use modular components, one per DFD process, so that:

- The scenario, travel-time, baseline, optimizer, and comparison parts can change independently;
- a third method (a different solver or objective weighting) can be added with no schema change; The weighting between travel time and unmet demand is a setting, not a value fixed in code;
- The solver can be switched between OR-Tools and PuLP;
- Sample data and scenarios can be updated easily.

3.5.5 Portability

The dashboard shall work in commonly used desktop and laptop browsers, and the backend shall run on a standard laptop. The same stack can be hosted on a free or low-cost plan for a demonstration.

4. Supporting Information

4.1 Algorithms

Baseline: greedy, nearest available base. The point of this method is to be simple and obvious, not clever.

1. Sort the affected areas by severity, from most severe to least severe.
2. For the current area, look up the travel time to every base.
3. Pick the nearest base that still has resources available, not simply the nearest base, since a closer one may already be empty.
4. Assign as many resources as the base can spare, up to what the area needs.
5. Subtract that amount from the base's remaining stock.
6. If the area's demand is still unmet, repeat from step 2 with the next-nearest base that has stock.
7. Move to the next area and repeat until all areas have been processed or resources run out.

Worked example:

Area	Demand	Base chosen	Base stock	Assigned	Still needed
A (Critical)	5	Base 1 (8 available)	8 to 3	5	0
B (High)	6	Base 1 (3 left)	3 to 0	3	3
B (continued)	3	Base 2 (4 available)	4 to 1	3	0

Area B draws from two bases because the nearest one ran dry. The weakness of the method is that it never looks ahead: it can spend a base's stock on a nearby area even though that stock would have been worth more in a more severe area it has not yet reached.

Optimized: constrained optimization: The model decides every assignment at once. For each base and each area, it chooses how many resources to send from that base to that area. It knows the travel time between every base and area, the stock each base holds, and the demand each area has. Two weights, one for travel time and one for unmet demand, are settings that decide how much each one counts.

The model looks for the plan with the lowest combined cost. That cost has two parts: the total travel time (the travel time of each trip multiplied by the number of resources sent on that trip) and the total unmet demand (the demand left over after everything is sent). Each part is multiplied by its weight, and the two are added together.

The plan must follow three rules:

- No base can send out more than the stock it holds.
- No area can receive more than it asked for.
- No quantity can be negative.

Because the model looks at the whole picture at once, it can hold back some of a base's stock for a more severe area, or serve one area partly from a second base when that yields a better total. This is also why it is slower than the baseline method and needs a real solver.

4.2 Evaluation Metrics

Both methods are measured identically. These numbers are the actual result of the project.

Metric	What it measures	Better when
Average response time	Mean travel time across all assignments	Lower
Maximum response time	The worst-served area's wait	Lower
Coverage ratio	Share of total demand that was met	Higher
Unmet demand	Total resources are short across all areas	Lower
Resource utilisation	Share of available resources actually deployed	Higher
Total travel cost	Sum of travel time across assignments	Lower

Metric	What it measures	Better when
Solver runtime	The time the method took to produce its answer	Lower

4.3 Limitations

- **The model is simplified.** It considers demand, capacity, and travel time. Real response also involves road damage, vehicle types, crew availability,y and hospital capacity.
- **Resource data is simulated.** Real base locations and stock levels are not public.
- **Public routing is not emergency routing.** OSRM gives times for ordinary traffic and rules, so travel times are systematically pessimistic for emergency vehicles.
- **Severity is an input, not a measurement.** The quality of the allocation depends on the Coordinator's judgment of severity.